

1. Funzionalità implementate

Regole complete · TUI · GUI · RMI · Socket · Classifica su DB · Partite multiple · Resilienza alle disconnessioni

2. Divisione del lavoro

Attività	Membri
UML, Sequence Diagram	Tutti
Sviluppo Model	Tutti
Database	Samuel
Rete	Francesco
GUI	Samuel, Mattia
TUI	Francesco, Gabriele
Card Factory	Gabriele
Test	Samuel, Gabriele
File JSON	Mattia, Samuel

3. Scelte implementative

Sicurezza e autenticazione (SHA-256)

Password hashate lato client con SHA-256 (64 char HEX). Non transitano mai in chiaro; il DB non espone credenziali nemmeno all'amministratore.

Beneficio: mitigazione sniffing e sicurezza contro attacchi al DB.

Gestione dinamica delle lobby

LobbyController permette spostamento libero tra stanze. In caso di abbandono dell'host, un altro utente viene promosso automaticamente. Lobby vuote vengono eliminate.

Beneficio: nessuna lobby vuota in memoria; flessibilità e user experience migliore per i giocatori.

Sicurezza del database

try-with-resources per chiusura automatica delle connessioni; PreparedStatement per SQL injection.

Beneficio: eliminazione memory leak, protezione da SQL injection.

Double Dispatch

Usato in GameAction.applyTo() e per l'aggiunta di carte alla Tribe, per risolvere il tipo dinamico a runtime senza instanceof/switch.

Beneficio: sfruttare polimorfismo, evitare pattern matching, codice leggibile ed estendibile.

DTO — Data Transfer Object

SnapshotBuilder costruisce periodicamente un MatchDTO con il solo stato necessario per le view (es. il deck non viene mai trasmesso).

Beneficio: semplice gestione dati da serializzare in rete, sicurezza dei dati riservati al model.

Pattern creazionali

Factory Method

CharacterCardFactory, EventCardFactory, BuildingCardFactory incapsulano la logica di creazione leggendo da file JSON.

Beneficio: nuovi tipi di carta senza modificare codice client; separazione parsing/logica; sorgente dati sostituibile anche per il testing.

Pattern strutturali

Flyweight

Le carte sono immutabili e allocate una sola volta all'avvio del server, poi condivise tra tutte le partite.

Beneficio: riduzione drastica memoria e I/O; minor traffico di rete per la serializzazione.

DAO

AccountDAO e MatchDAO astraggono tutti gli accessi al DB.

Beneficio: isolamento della logica di gioco dalla persistenza; sostituibilità del DB.

Pattern comportamentali

Strategy

BuildingEffect delega comportamenti variabili a strategie intercambiabili.

Beneficio: eliminazione switch-case; nuovi effetti senza modificare codice esistente; testing in isolamento.

Template Method

La gerarchia delle carte definisce metodi comuni e metodi specifici per ogni tipo.

Beneficio: riuso del codice condiviso; ogni sotto-classe rispetta il contratto della superclasse.

State

utilizzato per modellare il flusso di gioco tramite una FSM, permettendo di disaccoppiare il comportamento del MatchModel dalle regole di transizione tra le diverse fasi di gioco (sia automatiche che interattive).

Beneficio: gestione ordinata della logica di gioco; estensione di nuove fasi senza modificare quelle esistenti.

Visitor

Usato nella TUI per restituire una rappresentazione grafica unica per ogni tipo di carta/tessera.

Beneficio: separazione netta tra struttura dati e visualizzazione; possibilità a nuove modalità di rendering senza modificare le carte.

Command

Comandi client/server su Socket incapsulati in classi serializzabili.

Beneficio: eliminazione pattern matching lato server/client; comunicazione asincrona e bidirezionale; estensibilità tramite nuove command class.

4. Protocolli di comunicazione

Socket (TCP)

Basato sull'invio di oggetti serializzati. Sfrutta il Command Pattern per l'invio asincrono di richieste dal client e aggiornamenti dal server, con connessione persistente e bidirezionale.

RMI (Remote Method Invocation)

Il client invoca metodi esposti dal server (`VirtualServer`) in modo trasparente; il server utilizza le callback (`ClientRemote`) per notificare cambiamenti di stato. Integra ping/heartbeat per individuare cadute di connessione e innescare la logica di resilienza.

5. Copertura test

La fase di testing si è concentrata sul componente Model dell'applicazione, conseguendo una copertura dell'84% a livello di metodi (338 metodi risultano coperti su un totale di 398). Tale priorità è stata stabilita per isolare la logica di gioco rispetto a View e Controller. La batteria di test include inoltre verifiche volte a garantire che i meccanismi di creazione e configurazione iniziale delle entità di gioco da parte del pattern Factory avvengano in modo robusto, centralizzato ed esente da errori a runtime.